Credential cache file format

There are four versions of the file format used by the FILE credential cache type. The first byte of the file always has the value 5, and the value of the second byte contains the version number (1 through 4). Versions 1 and 2 of the file format use native byte order for integer representations. Versions 3 and 4 always use big-endian byte order.

After the two-byte version indicator, the file has three parts: the header (in version 4 only), the default principal name, and a sequence of credentials.

Header format

The header appears only in format version 4. It begins with a 16-bit integer giving the length of the entire header, followed by a sequence of fields. Each field consists of a 16-bit tag, a 16-bit length, and a value of the given length. A file format implementation should ignore fields with unknown tags.

At this time there is only one defined header field. Its tag value is 1, its length is always 8, and its contents are two 32-bit integers giving the seconds and microseconds of the time offset of the KDC relative to the client. Adding this offset to the current time on the client should give the current time on the KDC, if that offset has not changed since the initial authentication.

Principal format

The default principal is marshalled using the following informal grammar:

```
principal ::=
    name type (32 bits) [omitted in version 1]
    count of components (32 bits) [includes realm in version 1]
    realm (data)
    component1 (data)
```

On this page

Credential cache file format
Header format
Principal format
Credential format
Credential cache configuration entries

Table of contents

For users
For administrators
For application developers
For plugin module developers
Building Kerberos V5
Kerberos V5 concepts
Protocols and file formats
Credential cache file

```
    component2 (data)
    ...

data ::=
    length (32 bits)
    value (length bytes)
```

There is no external framing on the default principal, so it must be parsed according to the above grammar in order to find the sequence of credentials which follows.

Credential format

The credential format uses the following informal grammar (referencing the `principal` and `data` types from the previous section):

```
credential ::=
    client (principal)
    server (principal)
    keyblock (keyblock)
    authtime (32 bits)
    starttime (32 bits)
    endtime (32 bits)
    renew_till (32 bits)
    is_skey (1 byte, 0 or 1)
    ticket_flags (32 bits)
    addresses (addresses)
    authdata (authdata)
    ticket (data)
    second_ticket (data)

keyblock ::=
    enctype (16 bits) [repeated twice in version 3]
    data

addresses ::=
    count (32 bits)
    address1
    address2
    ...

address ::=
    addrtype (16 bits)
    data

authdata ::=
    count (32 bits)
    authdata1
    authdata2
    ...

authdata ::=
    ad_type (16 bits)
    data
```


format
Keytab file format
Replay cache file format
KDC cookie format
PKINIT freshness tokens
Kerberos Database (KDB) Formats
MIT Kerberos features
How to build this documentation from the source
Contributing to the MIT Kerberos Documentation
Resources

Full Table of Contents

Search

Go

There is no external framing on a marshalled credential, so it must be parsed according to the above grammar in order to find the next credential. There is also no count of credentials or marker at the end of the sequence of credentials; the sequence ends when the file ends.

Credential cache configuration entries

Configuration entries are encoded as credential entries. The client principal of the entry is the default principal of the cache. The server principal has the realm `X-CACHECONF:` and two or three components, the first of which is `krb5_ccache_conf_data`. The server principal's second component is the configuration key. The third component, if it exists, is a principal to which the configuration key is associated. The configuration value is stored in the ticket field of the entry. All other entry fields are zeroed.

Programs using credential caches must be aware of configuration entries for several reasons:

- A program which displays the contents of a cache should not generally display configuration entries.
- The ticket field of a configuration entry is not (usually) a valid encoding of a Kerberos ticket. An implementation must not treat the cache file as malformed if it cannot decode the ticket field.
- Configuration entries have an endtime field of 0 and might therefore always be considered expired, but they should not be treated as unimportant as a result. For instance, a program which copies credentials from one cache to another should not omit configuration entries because of the endtime.

The following configuration keys are currently used in MIT krb5:

fast_avail

: The presence of this key with a non-empty value indicates that the KDC asserted support for FAST (see **RFC 6113**) during the initial authentication, using the negotiation method described in **RFC 6806** section 11. This key is not associated with any principal.

pa_config_data

The value of this key contains a JSON object representation of parameters remembered by the preauthentication mechanism used during the initial authentication. These parameters may be used when refreshing credentials. This key is associated with the server principal of the initial authentication (usually the local krbtgt principal of the client realm).

pa_type

The value of this key is the ASCII decimal representation of the preauth type number used during the initial authentication. This key is associated with the server principal of the initial authentication.

proxy_impersonator

The presence of this key indicates that the cache is a synthetic delegated credential for use with S4U2Proxy. The value is the name of the intermediate service whose TGT can be used to make S4U2Proxy requests for target services. This key is not associated with any principal.

refresh_time

The presence of this key indicates that the cache was acquired by the GSS mechanism using a client keytab. The value is the ASCII decimal representation of a timestamp at which the GSS mechanism should attempt to refresh the credential cache from the client keytab.

start_realm

This key indicates the realm of the ticket-granting ticket to be used for TGS requests, when making a referrals request or beginning a cross-realm request. If it is not present, the client realm is used.

© Copyright 1985-2025, MIT.